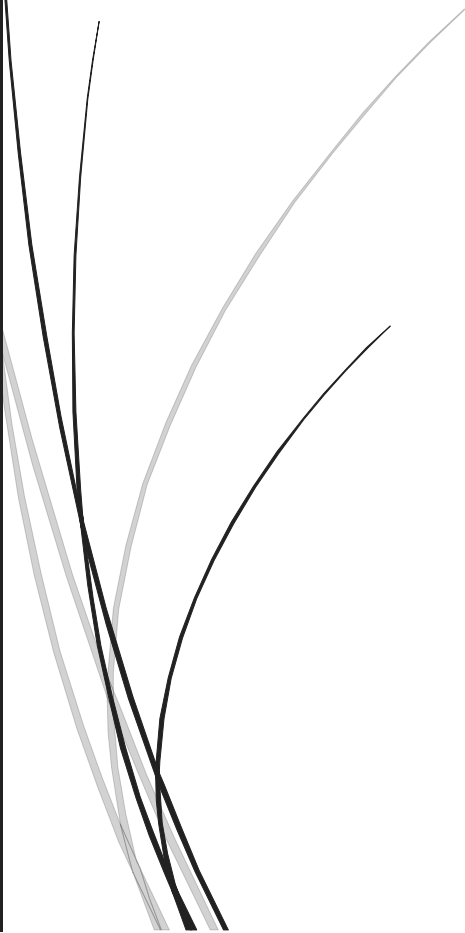




10/12/2025

RAPPORT : Génie Logiciel



Perso

DAHIR ALHUSSIN MAHMOUDET MEZIANE MILOUD

Table des matières

I) Introduction	2
II) Design Pattern.....	2
1) Architecture MVC et Synchronisation (Pattern Observer)	2
2) Pattern Strategy	4
3) Pattern Decorator	5
4) Pattern Factory	5
III) Analyse Éthique	6
1) Le manque de souplesse de l'outil.....	6
2) Le problème des parcours atypiques	7
IV) Stratégie de Tests	8
1) Tests Unitaires Automatisés	8
2) Tests Manuels	8
V) Conclusion.....	9

I) Introduction

Ce projet de conception logicielle avait pour but d'effectuer la ré-ingénierie d'un logiciel de sélection de candidats. Initialement, nous disposions d'une application mélangeant la logique métier et l'interface graphique, caractérisée par l'absence de contrôleur et un modèle assez pauvre (avec des classes comme `Applicant.java` pour simplement stocker les données). En conséquence, l'application était impossible à tester, difficile à maintenir et complexe à faire évoluer.

Notre objectif était donc de migrer cette base de code vers une architecture modulaire, robuste et conforme aux bonnes pratiques de conception (SOLID, GRASP). Nous avons mis en œuvre une architecture MVC stricte ainsi que plusieurs Design Patterns. Cela nous a permis de répondre aux exigences de maintenabilité et de testabilité, tout en améliorant le logiciel sur le plan éthique grâce à l'ajout de nouvelles fonctionnalités de filtrage.

II) Design Pattern

1) Architecture MVC et Synchronisation (Pattern Observer)

Au départ, l'application fonctionnait, mais son architecture posait problème : tout était mélangé. La logique métier (comme l'ajout ou la suppression de compétences) était codée directement à l'intérieur de la vue (VBox, boutons). Le modèle était faible (réduit à `Applicant.java` qui ne servait que de stockage), et il n'y avait pas de contrôleur. Concrètement, cela nous bloquait sur plusieurs points : impossible d'ouvrir deux fenêtres synchronisées (car chaque vue gardait ses données pour elle), impossibilité de tester la logique sans lancer l'interface graphique, et difficulté à maintenir le code sur le long terme ou à ajouter de nouvelles fonctionnalités.

Nous avons donc séparé les responsabilités en trois blocs distincts, c'est-à-dire l'architecture MVC, reliés par le pattern Observer :

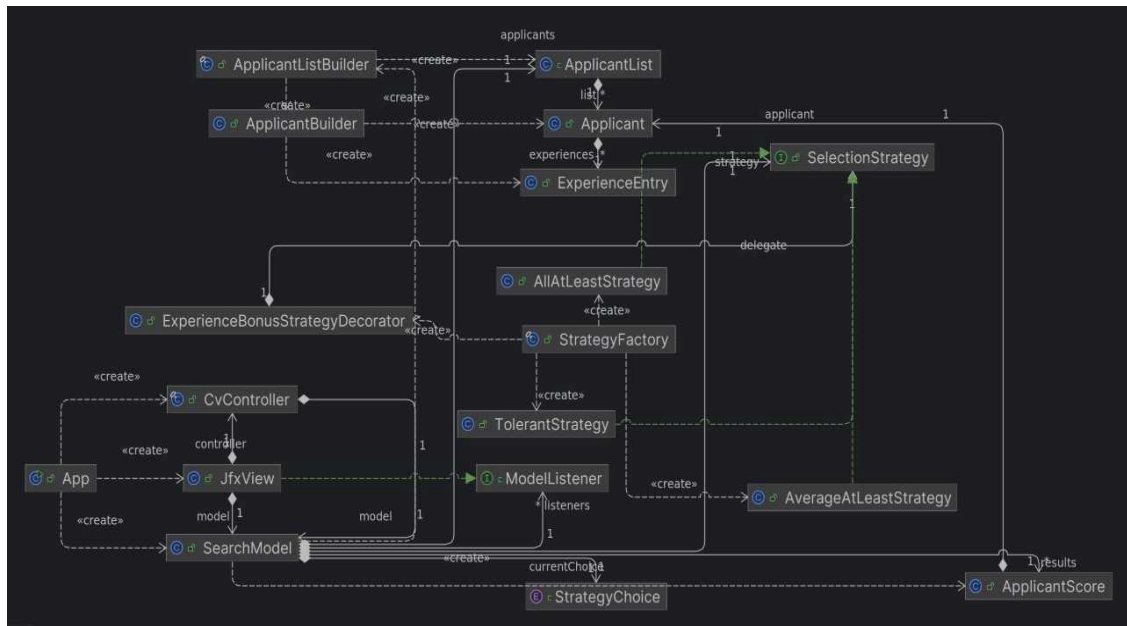
- Le Modèle (`SearchModel`) : C'est le cerveau de l'application. Contrairement au début, nous l'avons amélioré en un Modèle Riche. Il ne se contente pas de stocker la liste des candidats, c'est lui qui contient toute la logique : ajouter une compétence, vérifier les doublons, trier les résultats. Il est Observable : il prévient les autres quand il change, mais il ne sait pas qui l'écoute.
- La Vue (`JfxView`) : Elle est devenue totalement passive. Elle ne fait aucun calcul. Elle implémente l'interface `ModelListener` et attend simplement que le modèle lui dise "J'ai changé" pour mettre la vue à jour.
- Le Contrôleur (`CvController`) : Nous l'avons conçu en mode Push-based. Son rôle est simple : il intercepte les clics de l'utilisateur et pousse les ordres vers le modèle (`addRequiredSkill`, `search`). C'est pertinent ici car cela évite au modèle

de devoir surveiller le contrôleur en permanence, ce qui économise des ressources.

En conséquence, cette séparation a eu plusieurs impacts directs sur notre façon de développer et sur la qualité du logiciel :

- Synchronisation immédiate : Dans notre classe App, on lance deux fenêtres (JfxView) sur le même SearchModel. Dès qu'on ajoute un skill dans l'une, le modèle notifie tout le monde, et l'autre fenêtre se met à jour instantanément.
- Portabilité et Indépendance : Le couplage entre la logique et l'interface est maintenant très faible. Cela rend notre application beaucoup plus portable. On pourrait tout à fait imaginer, dans un second temps, créer une interface Web pour ce logiciel. On garderait 100% de notre SearchModel et de nos tests, on aurait juste à remplacer la JfxView par une page HTML. Le cœur du métier ne changerait pas.
- Facilité d'extension : L'architecture a rendu l'ajout de fonctionnalités beaucoup plus sûr. Par exemple, pour ajouter la suppression d'une compétence (le bouton croix), nous n'avons pas eu besoin de fouiller dans tout le code. On a ajouté la méthode dans le modèle, le bouton dans la vue, et c'était fini. On ne risque pas de casser une autre partie de l'appli en touchant à la vue. C'est exactement la même chose pour le bouton Clear que nous avons ajouté pour supprimer tous les skills d'un coup au lieu de les supprimer un par un : il a seulement suffi de créer la fonction dans le modèle et de la relier au bouton, sans complexité supplémentaire.
- Travail en équipe et Tests : Le MVC a facilité la répartition des tâches dans le binôme. L'un pouvait travailler sur l'amélioration de la Vue pendant que l'autre peaufinait les algorithmes d'ajout de skill dans le Modèle sans se gêner. Enfin, cela a débloqué les tests unitaires : nous avons pu tester toute la logique de SearchModel (ajout, calculs) de manière indépendante, sans jamais avoir besoin de lancer une fenêtre JavaFX, ce qui rend les tests plus rapides et plus fiables.

Diagramme des classes :



2) Pattern Strategy

Ce pattern a été intégré lors de la phase de ré-ingénierie et s'est imposé comme une évidence face au blocage du code initial. Avant notre intervention, la décision de garder ou non un candidat était figée et mélangée à l'intérieur du modèle principal. Si nous avions voulu gérer plusieurs méthodes sans ce pattern, nous aurions été contraints d'écrire une longue suite de conditions du type `if (stratégie == 1)` directement dans le cœur de l'application. Cette approche posait un problème de sécurité du code car pour ajouter une nouvelle règle, comme une moyenne, il aurait fallu modifier le fichier principal avec le risque de casser ce qui fonctionnait déjà.

Pour mieux comprendre l'intérêt de ce découpage, on peut prendre l'exemple du paiement dans un magasin. Le montant à payer reste le même, mais le traitement est totalement différent si le client choisit la carte bancaire ou les espèces. Le

logiciel de caisse doit pouvoir basculer d'une méthode à l'autre instantanément sans tout reprogrammer. Notre application fonctionne exactement sur ce principe, le processus global reste le même mais la logique de calcul de la note change radicalement selon qu'on choisisse une stratégie stricte ou une moyenne.

Dans notre cas, ce pattern est devenu indispensable dès qu'on a ajouté la liste déroulante (ComboBox) dans l'interface. Au lieu d'avoir des `if` partout, chaque méthode de recrutement est maintenant isolée dans une classe qui respecte l'interface `SelectionStrategy`. Cette interface impose deux méthodes simples : `isSelected` pour dire "oui" ou "non" au candidat, et `computeScore` pour lui donner une note.

Ce qui est intéressant techniquement, c'est que le changement se fait à la volée. Le `SearchModel` possède une variable de type `SelectionStrategy`. Quand l'utilisateur change l'option dans le menu, le contrôleur met simplement une nouvelle stratégie dans cette variable. Le modèle n'a pas besoin de redémarrer : il continue son travail en utilisant le nouvel objet qu'on vient de lui donner pour faire ses calculs.

L'avantage principal, c'est la clarté. Chaque règle (Moyenne, Seuil strict) a son propre fichier au lieu d'être mélangée dans une fonction géante. Cela nous évite les if / else if à rallonge qui rendent le code illisible. C'est d'ailleurs comme ça qu'on a pu ajouter facilement notre stratégie "Expert" (Tout > 80%) destinée aux profils seniors, ainsi que notre stratégie éthique (TolerantStrategy, que l'on expliquera dans la partie éthique), en créant juste de nouvelles classes sans toucher au reste du code qui fonctionnait déjà.

3) Pattern Decorator

Nous avons décidé d'ajouter ce pattern lors de l'implémentation de la valorisation de l'expérience professionnelle. L'objectif était d'ajouter un bonus par année d'expérience sur une compétence précise.

Cependant, nous voulions appliquer ce bonus quelle que soit la stratégie choisie, que ce soit "Moyenne" ou "Tout >= 50". Sans ce pattern, en utilisant un héritage classique, nous aurions été obligés de créer une classe pour chaque combinaison possible : MoyenneAvecExperience, SeuilAvecExperience, TolerantAvecExperience, etc. On se serait retrouvés avec des dizaines de classes et du code dupliqué partout.

Notre solution a donc été d'emballer nos stratégies avec le pattern Decorator, qui permet d'ajouter des fonctionnalités à un objet sans le modifier de l'intérieur. Concrètement, nous avons créé la classe ExperienceBonusStrategyDecorator.

Pour le reste de l'application, ce décorateur ressemble à une stratégie normale (il implémente la même interface SelectionStrategy), mais à l'intérieur, il fonctionne différemment : il possède une référence vers une autre stratégie (le delegate) qu'il "enveloppe".

Le fonctionnement est le suivant : quand on demande le score au décorateur via la méthode computeScore, il procède en deux étapes :

- D'abord, il demande à la stratégie qu'il contient (par exemple la "Moyenne") de calculer la note de base.
- Ensuite, il calcule le bonus d'expérience de son côté et l'ajoute au résultat final.

L'avantage concret est que cette approche nous offre une flexibilité totale. Au lieu de tout avoir dans les mêmes classes, on peut désormais appliquer le bonus à n'importe quelle règle de recrutement, même celles qu'on créera dans le futur. De plus, cela sépare bien les responsabilités : le calcul de la moyenne reste dans sa classe, et le calcul du bonus est isolé dans le décorateur. On peut ainsi modifier la façon dont on calcule l'expérience sans risquer de casser le calcul de la moyenne.

4) Pattern Factory

Une fois les patterns Strategy et Decorator en place, nous nous sommes heurtés à un problème pratique : la création des objets. Dans notre contrôleur, nous récupérons le choix de l'utilisateur via l'interface, mais nous ne voulions surtout pas instancier directement les stratégies avec des new à l'intérieur du contrôleur.

Cela aurait créé un couplage fort, mais surtout, cela aurait complexifié le code inutilement. C'est particulièrement vrai pour le décorateur : pour créer une stratégie "Moyenne avec Expérience", il ne suffit pas d'un simple new. Il faut instancier la stratégie de base, puis l'emballer dans le décorateur avec le bon coefficient. Écrire toute cette logique d'assemblage au milieu du contrôleur aurait rendu le code lourd et difficile à lire.

Pour résoudre cela, nous avons utilisé une simple Factory (StrategyFactory). Afin de sécuriser les échanges entre la vue et cette fabrique, nous avons introduit une énumération StrategyChoice qui liste toutes les options disponibles (comme AVG_50 ou TOLERANT). L'utilisation de cet Enum est bien plus robuste que de simples chaînes de caractères car elle empêche les erreurs de frappe et permet de remplir la liste déroulante de l'interface automatiquement.

Concrètement, la Factory agit comme une boîte noire qui reçoit ce choix standardisé en entrée et renvoie l'objet SelectionStrategy correspondant, entièrement configuré et prêt à l'emploi. L'intérêt s'est vraiment vu lors de l'intégration du bonus d'expérience : nous avons pu centraliser toute la complexité de l'assemblage dans la Factory. Le contrôleur, lui, reste propre : il appelle juste la Factory sans savoir comment les objets sont construits derrière.

III) Analyse Éthique

Avec l'avancée de l'IA, les entreprises ont développé de nouveaux outils pour gagner du temps sur l'analyse des CV : les ATS. En effet, pour économiser du temps et de l'argent, les recruteurs effectuent désormais une première analyse automatique pour filtrer les candidats selon des mots-clés et l'expérience, avant de transmettre les profils retenus à une personne réelle pour la suite du processus de recrutement.

Le développement de notre logiciel nous a montré beaucoup de manques dans nos stratégies initiales, ce qui nous a menés à nous poser plusieurs questions pour tenter d'obtenir le logiciel le plus éthique possible.

Nous allons donc, dans un premier temps, analyser la rigidité de l'outil à ses débuts et nos solutions pour l'améliorer. Ensuite, nous aborderons le problème de certains parcours atypiques et les soucis d'équité que cela pose, avant de terminer par une conclusion.

1) Le manque de souplesse de l'outil

Le problème principal d'un algorithme, c'est qu'il applique les règles mathématiques à la lettre, sans réfléchir au contexte. En développant et en testant nos stratégies, nous avons identifié deux cas où cette rigidité posait problème :

- Le problème de la majuscule : Au début, notre recherche était sensible à la majuscule. Concrètement, si un recruteur cherchait la compétence "Java" et qu'un candidat avait écrit "java" (en minuscule) dans son CV, son score tombait à 0. C'est injuste d'éliminer un bon profil pour une simple différence de majuscule ou de formatage.
- La sévérité aveugle : Nous avons implémenté une stratégie "Expert" (Tout > 80%) qui est très utile pour recruter des profils seniors. Cependant, nous nous sommes rendu compte que si on appliquait ce même moule strict à tout le monde (notamment des juniors ou des stagiaires), on éliminait d'excellents candidats qui avaient juste une note légèrement en dessous du seuil.

Pour pallier cela, nous avons d'abord rendu la recherche robuste en modifiant le code pour utiliser `equalsIgnoreCase`, rendant le tri insensible aux majuscules. Surtout, nous avons créé la `TolerantStrategy`. C'est le contrepoids éthique de la stratégie "Expert" : elle imite un recruteur humain capable d'indulgence, en acceptant un candidat même s'il a une faiblesse (une note sous la moyenne), tant que son dossier global reste solide.

2) Le problème des parcours atypiques

Notre volonté initiale était de valoriser l'ancienneté. Dans le recrutement, on part souvent du principe qu'un candidat avec beaucoup d'années de pratique est plus compétent ou plus autonome. C'est pour cela que nous avons codé un système de bonus via le Décorateur : pour chaque année d'expérience trouvée dans le CV, l'algorithme ajoute automatiquement 0.2 point à la note finale. L'objectif était donc positif : aider le recruteur à faire ressortir les profils "seniors".

Cependant, ce système mathématique peut se révéler injuste. En ne regardant que le compteur d'années, on favorise mécaniquement les carrières linéaires (c'est-à-dire continues et sans interruption). Le souci, c'est que cela pénalise les candidats qui ont des "trous" dans leur CV. Dans la réalité, cela touche statistiquement plus les femmes (congé maternité, charge familiale) ou les personnes ayant eu des accidents de la vie (maladie, aidants). L'algorithme ne cherche pas à comprendre *pourquoi* il y a une interruption, il attribue juste une note plus basse parce qu'il manque des années.

C'est un problème célèbre dans le monde de la Tech, illustré par le scandale d'Amazon en 2015. Leur intelligence artificielle de recrutement avait appris à pénaliser les CV féminins car elle se basait sur l'historique de l'entreprise (très masculin) et rejetait les profils qui s'écartaient de la norme. (Source : <https://www.reuters.com/article/us-amazon-com-jobs-automation-insight-idUSKCN1MK08G>).

Notre bonus d'expérience, s'il est utilisé comme seul critère, reproduit exactement ce biais en désavantageant les profils atypiques.

Pour conclure, cet outil doit rester une aide pour gagner du temps face à de gros volumes de CV, mais c'est l'humain qui doit avoir le dernier mot. Un algorithme ne peut pas saisir toutes les nuances d'un parcours de vie.

C'est pour cette raison qu'il était crucial techniquement de ne pas imposer une seule règle à tout le monde. En offrant le choix entre une stratégie stricte ("Expert" > 80%) pour des besoins pointus et une stratégie souple ("Tolérant") pour laisser une chance aux profils atypiques, nous remettons la décision finale entre les mains du recruteur, plutôt que de laisser la machine exclure aveuglément.

IV) Stratégie de Tests

Pour garantir la robustesse de l'application et sécuriser le refactoring, nous avons adopté une stratégie de test complète divisée en deux catégories.

1) Tests Unitaires Automatisés

L'architecture MVC nous a permis de tester toute la logique métier sans avoir besoin de lancer l'interface graphique, ce qui rend les tests très rapides. Nous avons concentré nos efforts sur les points critiques. D'abord la robustesse avec ApplicantTest où nous avons ajouté un test spécifique nommé testCaseInsensitiveSkillSearch qui prouve que notre correction fonctionne et que "Java" et "java" sont bien reconnus comme identiques. Ensuite la logique mathématique avec StrategyTest qui vérifie que les calculs de moyennes sont justes et surtout que notre Décorateur ajoute bien le bonus prévu sans erreur. Nous y testons aussi la TolerantStrategy pour prouver qu'elle repêche bien les candidats limites. Enfin, nous avons sécurisé la création des objets avec StrategyFactoryTest pour s'assurer que chaque option du menu génère bien la bonne classe Java correspondante.

2) Tests Manuels

Comme certaines interactions graphiques sont complexes à automatiser, nous avons validé manuellement les fonctionnalités liées à l'interface. Le test le plus important a été celui de la synchronisation MVC. Le protocole était simple : lancer l'application qui ouvre deux fenêtres, ajouter une compétence comme "C++" dans la première, et vérifier visuellement son apparition immédiate dans la seconde. Nous avons également validé le tri visuel de la liste pour s'assurer que les meilleurs candidats apparaissent bien en haut de la liste avec leurs scores détaillés.

V) Conclusion

Au final, ce projet nous a permis de comprendre concrètement l'intérêt d'une architecture propre. Le passage d'un code monolithique à une structure MVC stricte a transformé une application fragile en un logiciel modulaire où l'on peut ajouter des fonctionnalités sans peur de tout casser. Le projet rendu respecte l'ensemble des standards de qualité demandés, il ne contient aucune erreur Checkstyle, et l'intégration continue (CI) est totalement fonctionnelle sur GitLab. Nous avons réussi à combiner des exigences techniques fortes comme les Design Patterns avec une réflexion éthique concrète sur le recrutement, prouvant que le code peut aussi apporter des solutions à des problèmes humains.